ESPRIT · Plateforme WEB DISTRIBUÉES ·

AGCP

Constitution automatique de groupes d'apprentissage collaboratif & suivi analytique

Validation 1 · Conception et modélisation microservices



6 microservices

Classe : 4DAS-I10 G1· 2026–2027

Plan de la présentation

01 **Présentation de l'équipe**

Les cinqs membres du groupe AGCP

02 **Contexte et problématique**

Pourquoi automatiser la constitution des groupes

03 **Acteurs et fonctionnalités**

Étudiant, enseignant, administrateur

04 **Découpage en microservices**

6 services, une base par service

04 **Architecture et communication**

Gateway, Eureka, Config, Keycloak, RabbitMQ

06 **Modélisation des données**

Diagramme de domaine et de classes

07 **Présentation de chaque microservice**

Classes, endpoints REST, interactions

08 **conclusion**

Synthèse de la conception et prochaines étapes

Présentation de l'équipe



Med Bachar Mhamed



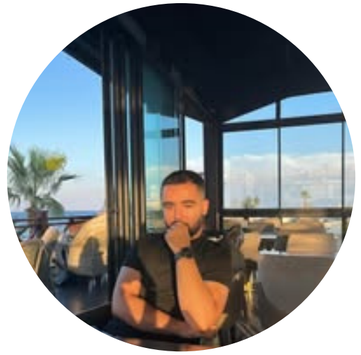
Maram Balti



Mohamed Elleuch



Louay Ben Rayena



Amen Bannour

Contexte et problématique



Libre choix

Reproduit les cercles d'amis : certains groupes concentrent les meilleurs profils, d'autres s'enlisent.



Composition manuelle

Lente, impraticable au-delà de quelques dizaines d'étudiants et exposée aux biais.



Notre solution : AGCP

Profil multi-critères + clustering : des groupes diversifiés à l'intérieur et comparables entre eux, validés par l'enseignant.

Objectivité : mêmes règles pour tous

Équité : groupes de niveau comparable

Gain de temps : une promotion en secondes

Acteurs et fonctionnalités



Étudiant

- Complète son profil et le questionnaire
- Découvre son groupe et son équipe
- Choisit un avatar, gagne des badges
- Suit sa contribution



Enseignant

- Crée projets et classes
- Lance une constitution avec critères pondérés
- Valide et ajuste les groupes proposés
- Suit les indicateurs de qualité



Administrateur

- Gère les comptes et les rôles
- Consulte les tableaux de bord agrégés
- Compare ML vs répartition aléatoire

Prioritaire : profil, import CSV, constitution, indicateurs, validation · Ensuite : badges, classement par équipe, tableaux de bord

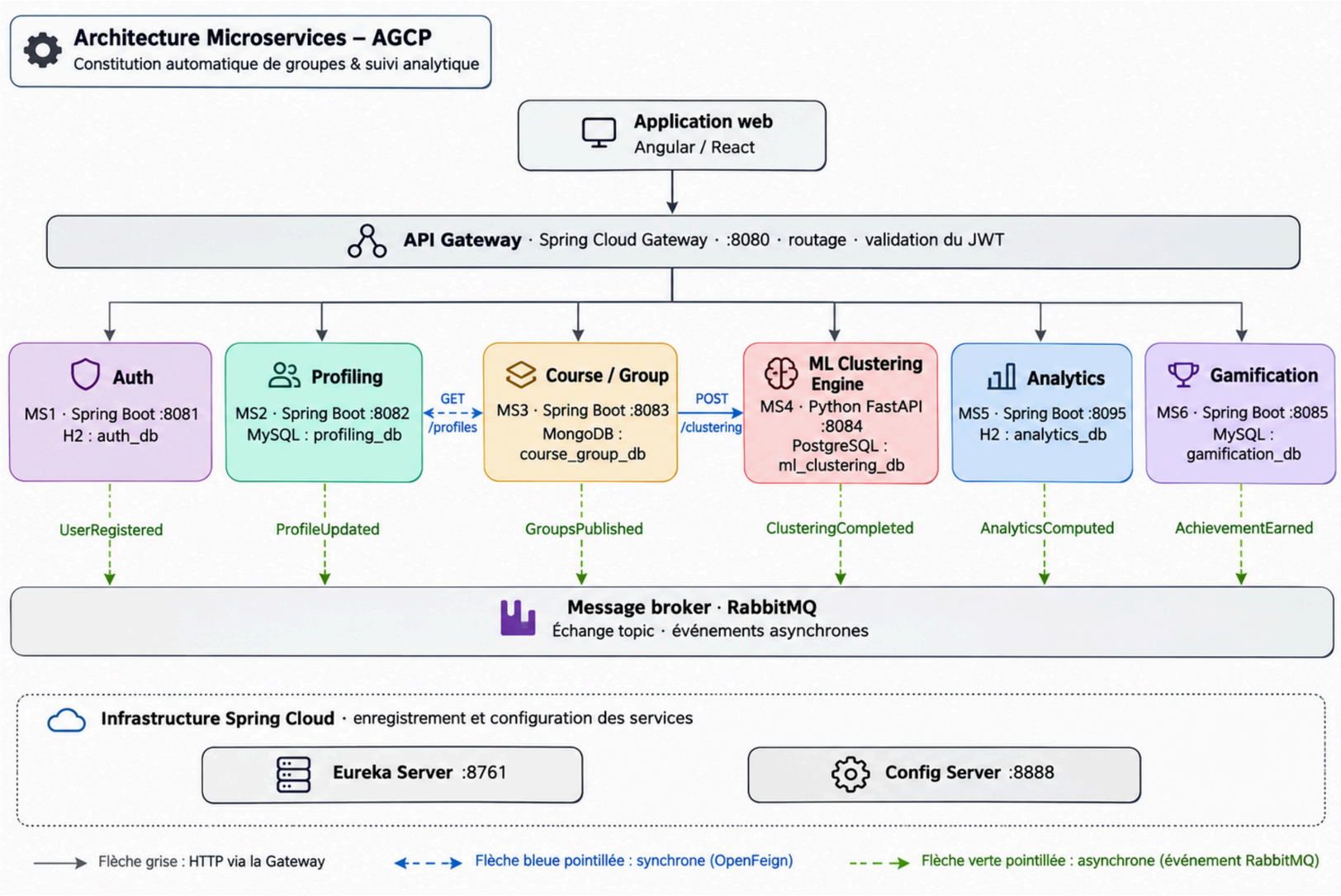
Découpage en microservices

Un microservice par capacité métier, chacun avec sa propre base : aucune base partagée.

#	Microservice	Responsable	Technologie	Port	Moteur BD	Nom de la base
MS1	**Auth Service**	Louay Ben Rayena	Spring Boot	:8081	H2	`auth_db`
MS2	**User / Profiling Service**	Mohamed Elleuch	Spring Boot	:8082	MySQL	`profiling_db`
MS3	**Course / Group Service**	Maram Balti	Spring Boot	:8083	MongoDB	`course_group_db`
MS4	**ML Clustering Engine**	equipe	Python · FastAPI	:8084	PostgreSQL	`ml_clustering_db`
MS5	**Gamification Engine**	Med Bachar Mhamed	Spring Boot	:8085	MySQL	`gamification_db`
MS6	**Analytics Service**	Amen Bannour	Spring Boot	:8095	H2	`analytics_db`

Infrastructure commune : API Gateway :8080 · Eureka :8761 · Config Server :8888 · Keycloak · RabbitMQ · Docker Compose

Architecture technique



Infrastructure

API Gateway :8080
point d'entrée, routage, JWT

Eureka :8761
enregistrement et découverte

Config Server :8888
configuration centralisée

Keycloak
OAuth2 / OIDC, rôles

RabbitMQ
échange topic, événements

Docker Compose
un conteneur par composant

Communication entre microservices

Synchrone · OpenFeign

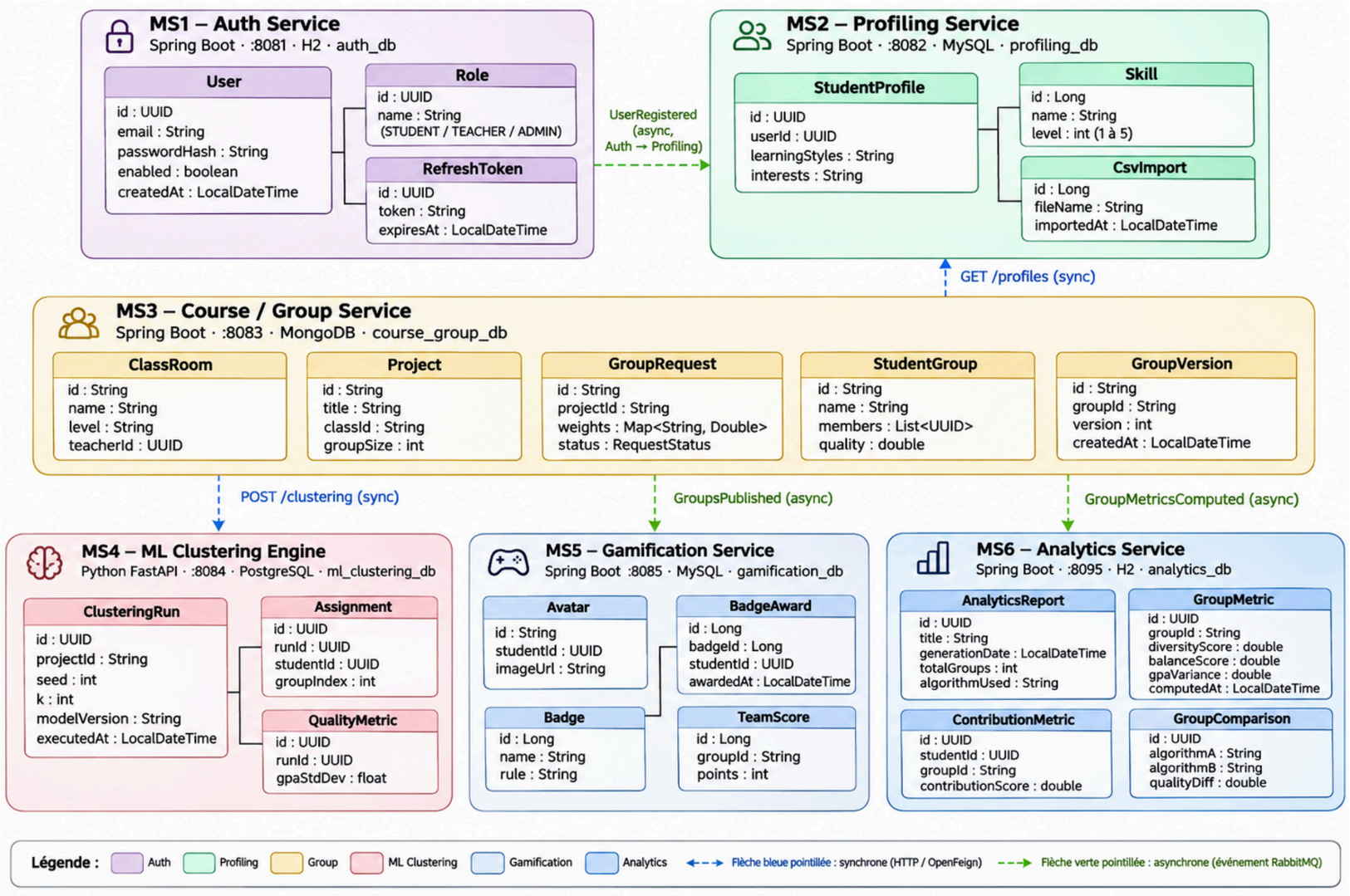
Appel	Endpoint	Pourquoi
MS3 → MS2	GET /api/profiles/{id}	profils des membres
MS3 → MS4	POST /api/ml/clusters	lancer le clustering
MS4 → MS2	GET /api/profiles/vectors	vecteurs pour le ML
MS6 → MS3	GET /api/groups	groupes à analyser
MS6 → MS2	GET /api/profiles	GPA et compétences

Asynchrone · RabbitMQ

Événement	Émetteur → abonnés	Effet
UserRegistered	MS1 → MS2	profil vide créé
GroupsPublished	MS3 → MS5, MS6	équipe + indicateurs
ClusteringCompleted	MS4 → MS3	proposition prête
AchievementEarned	MS5 → notifications	badge obtenu
AnalyticsComputed	MS6 → front-end	tableaux de bord

Si un service appelé est indisponible : circuit breaker (Resilience4j) et message d'erreur clair ; les événements restent dans RabbitMQ jusqu'à leur traitement.

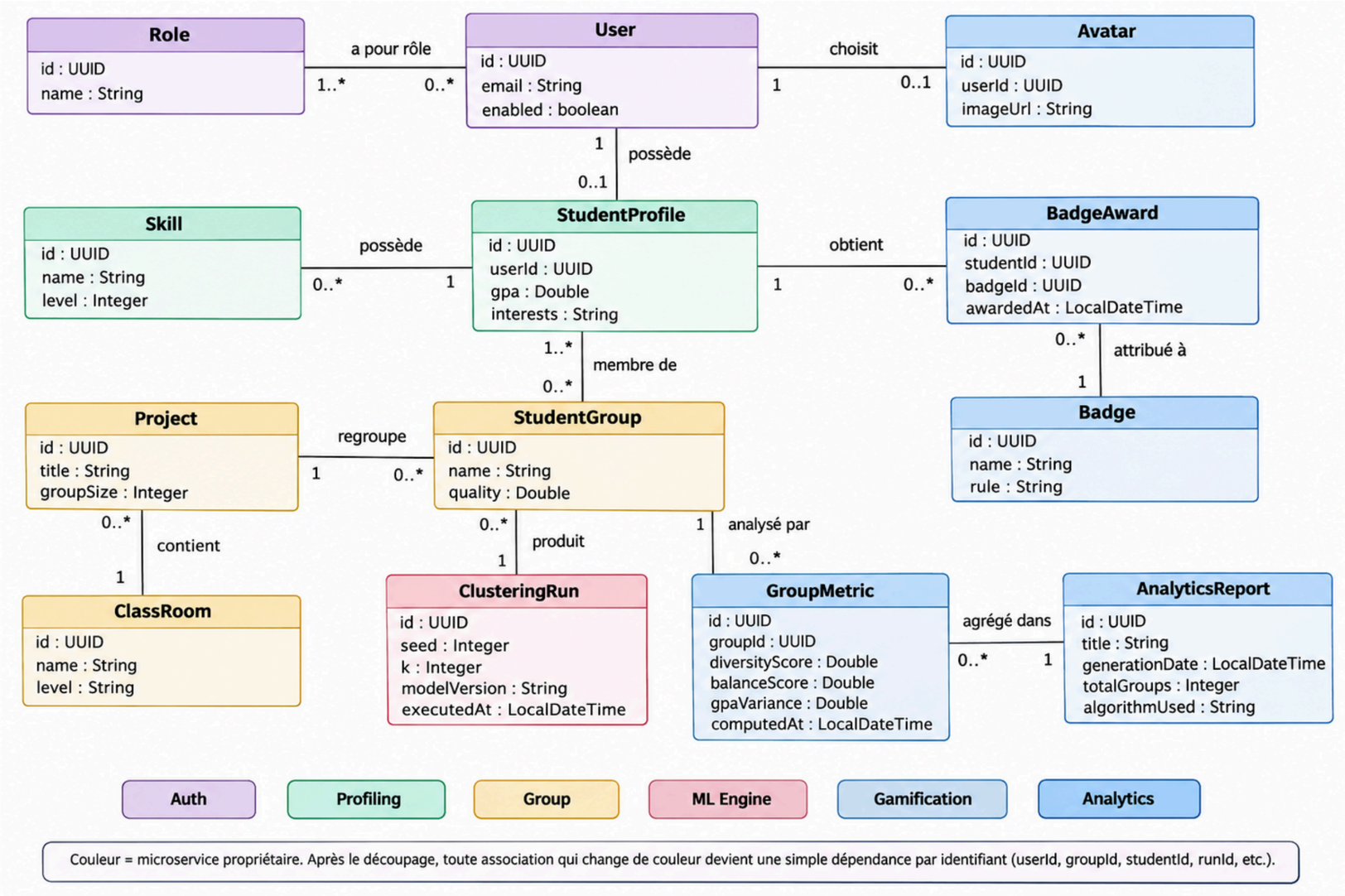
Modèle de données par microservice



À retenir

- Chaque cadre = un microservice et sa base
- Les entités ne sont jamais partagées entre services
- Flèches bleues : appels synchrones (OpenFeign)
- Flèches vertes : événements RabbitMQ

Diagramme de classes global



À retenir

- Couleur = microservice propriétaire
- Une association qui change de couleur devient une référence par identifiant (userId, groupId, runId)
- Cohérence assurée par les API et les événements

Présentation de chaque microservice

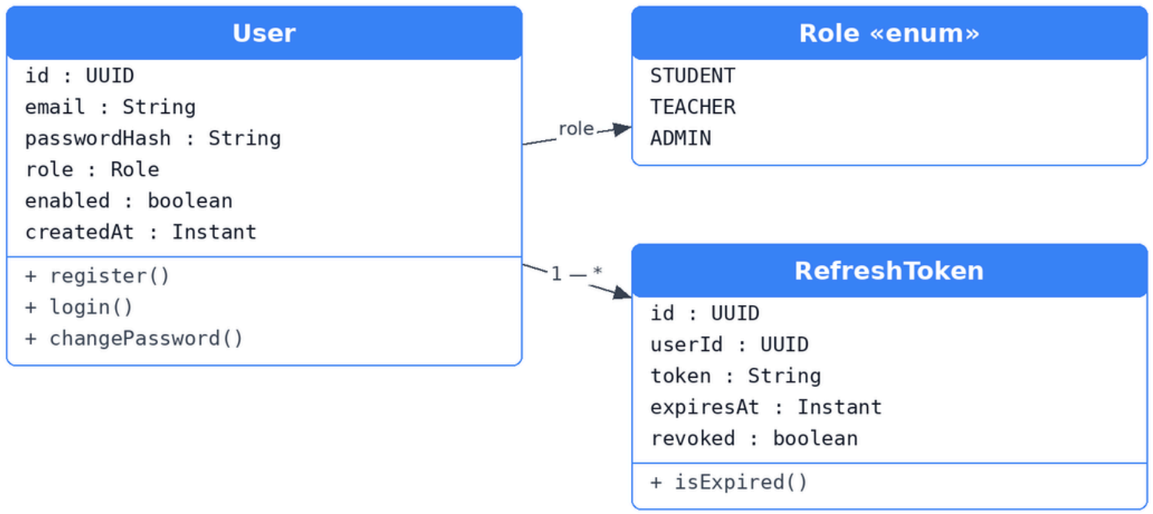
MS1	MS2	MS3	MS4	MS5	MS6
Auth	**Profiling**	**Course / Group**	**ML Clustering**	**Gamification**	**Analytics**
Louay Ben Rayena	Mohamed Elleuch	Maram Balti	Equipe	Med Bachar Mhamed	Amen Bannour

MS1 · Auth Service

MS1

Responsable : Louay Ben Rayena Spring Boot · port 8081 · H2 (auth_db)

Diagramme de classes



Endpoints REST

POST	/api/auth/register	Inscription
POST	/api/auth/login	Connexion → JWT
POST	/api/auth/refresh	Renouveler le token
POST	/api/auth/logout	Révoquer le token
GET	/api/auth/me	Utilisateur courant
GET	/api/users	Liste (ADMIN)
PUT	/api/users/{id}/role	Changer le rôle (ADMIN)

Interactions (autres services)

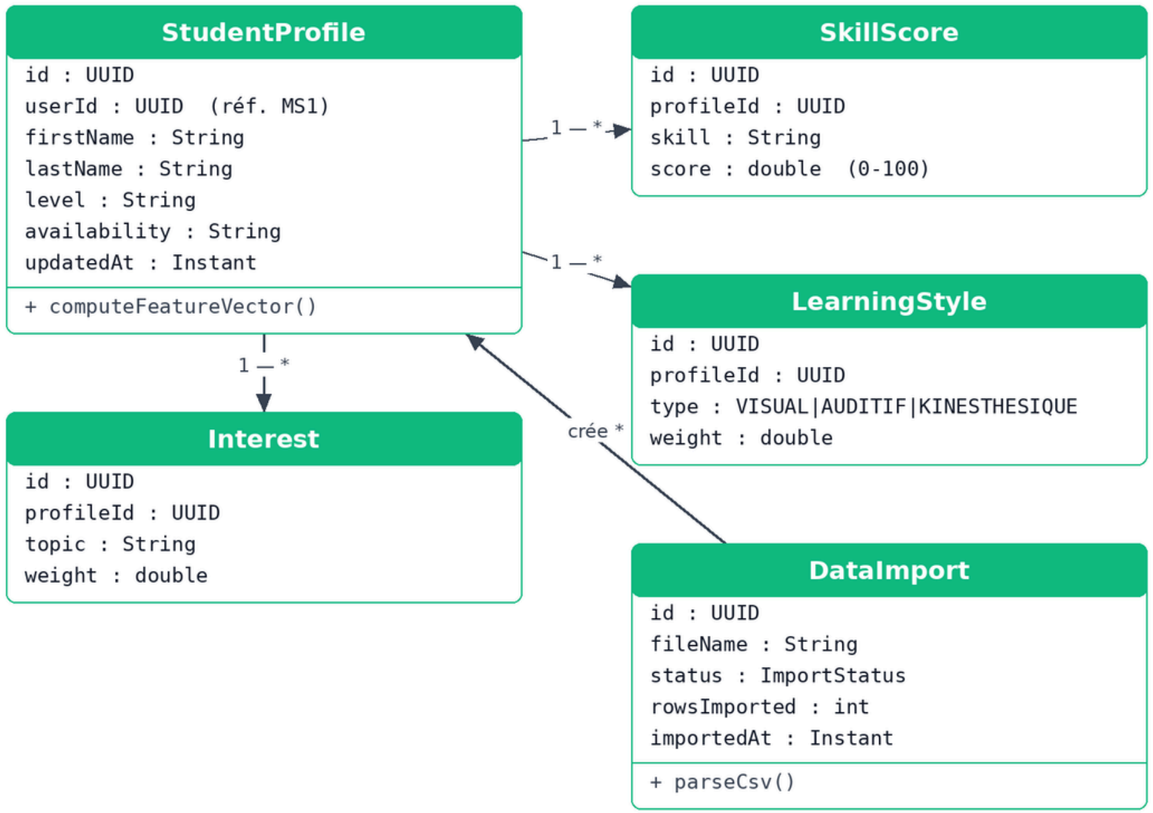
Publie	Événement UserRegistered → RabbitMQ (profiling crée le profil)
Publie	Événement UserDeleted → RabbitMQ
REST / Feign	Gateway :8080 valide le JWT émis par ce service / Keycloak
Infra	S'enregistre dans Eureka :8761, config via Config Server :8888

MS2 · User / Profiling Service

MS2

Responsable : [Mohamed Elleuch] Spring Boot · port 8082 · MySQL (profiling_db)

Diagramme de classes



Endpoints REST

GET	/api/profiles/{userId}	Lire un profil
PUT	/api/profiles/{userId}	Modifier un profil
POST	/api/profiles/{userId}/skills	Ajouter une compétence
PUT	/api/profiles/{userId}/learning-style	Style d'apprentissage
POST	/api/profiles/{userId}/interests	Ajouter un intérêt
POST	/api/profiles/import	Import CSV (TEACHER)
GET	/api/profiles/vectors?ids=	Vecteurs pour le ML

Interactions (autres services)

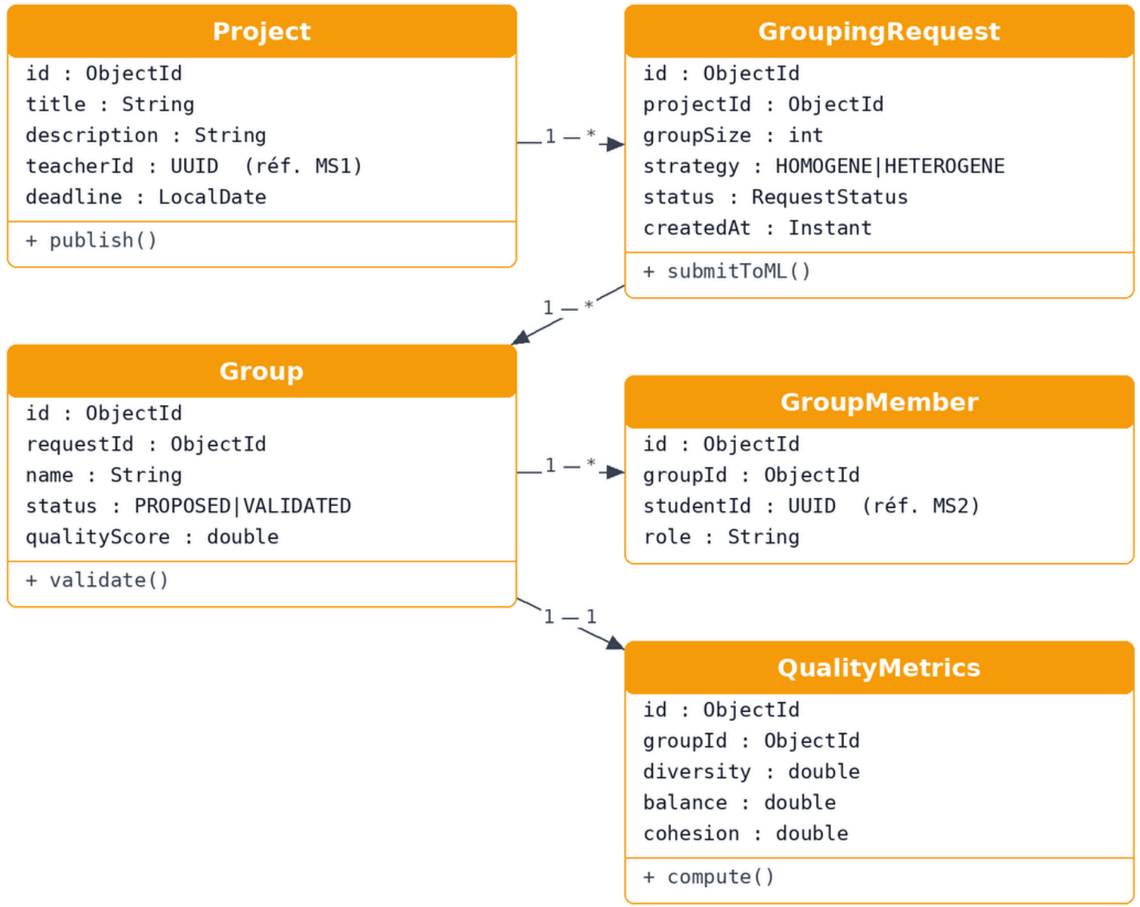
Consomme	Consomme UserRegistered (RabbitMQ) → crée un StudentProfile vide
REST / Feign	OpenFeign ← ml-clustering : GET /api/profiles/vectors
REST / Feign	OpenFeign ← group-service : GET /api/profiles/{id}
Infra	Eureka :8761 + Config Server :8888

MS3 · Course / Group Service

MS3

Responsable : Maram Balti Spring Boot · port 8083 · MongoDB (course_group_db)

Diagramme de classes



Endpoints REST

POST	/api/projects	Créer un projet (TEACHER)
GET	/api/projects	Lister les projets
POST	/api/grouping-requests	Lancer la formation de groupes
GET	/api/grouping-requests/{id}	Statut de la demande
GET	/api/groups?projectId=	Groupes d'un projet
PUT	/api/groups/{id}/validate	Valider un groupe
PUT	/api/groups/{id}/members	Ajuster les membres
GET	/api/groups/{id}/quality	Indicateurs de qualité

Interactions (autres services)

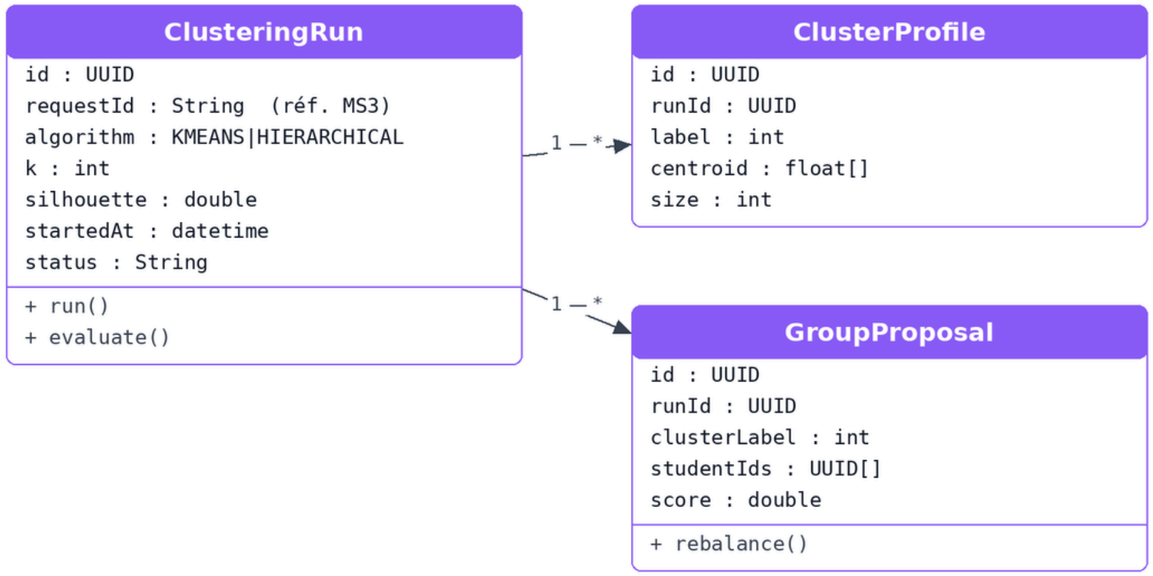
REST / Feign	OpenFeign → ml-clustering : POST /api/ml/clusters
REST / Feign	OpenFeign → profiling : GET /api/profiles/{id}
Publie	Événement GroupValidated → RabbitMQ (gamification crée l'identité d'équipe)
Infra	Eureka :8761 + Config Server :8888

MS4 · ML Clustering Engine

MS4

Responsable : Equipe Python · FastAPI · port 8084 · PostgreSQL (ml_clustering_db)

Diagramme de classes



Endpoints REST

POST	`/api/ml/clusters`	Lancer un clustering
GET	`/api/ml/runs/{id}`	Résultat d'un run
GET	`/api/ml/runs/{id}/proposals`	Groupes proposés
POST	`/api/ml/runs/{id}/rebalance`	Rééquilibrer
GET	`/api/ml/algorithms`	Algorithmes disponibles
GET	`/health`	Health check (Eureka)

Interactions (autres services)

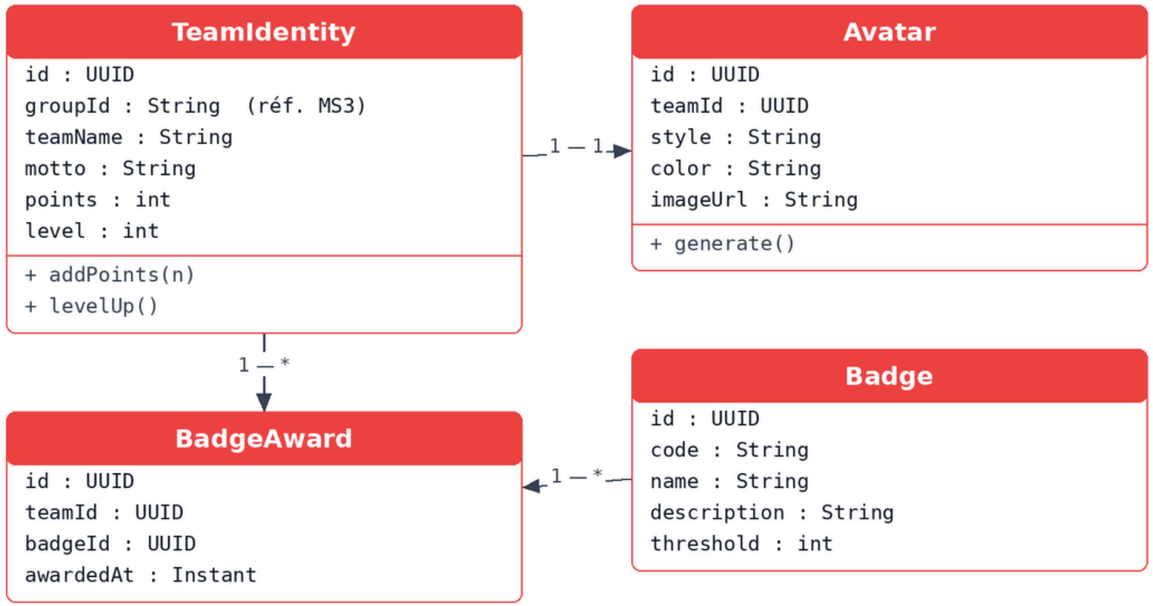
REST / Feign	REST → profiling : GET /api/profiles/vectors (features étudiants)
REST / Feign	REST ← group-service : POST /api/ml/clusters
Publie	Événement ClusteringCompleted → RabbitMQ
Infra	Enregistrement Eureka via py-eureka-client

MS5 · Gamification Engine

MS5

Responsable : Med Bachar Mhamed Spring Boot · port 8085 · MySQL (gamification_db)

Diagramme de classes



Endpoints REST

GET	/api/gamification/teams/{groupId}	Identité d'équipe
PUT	/api/gamification/teams/{groupId}	Personnaliser (nom, devise)
POST	/api/gamification/teams/{groupId}/avatar	Générer l'avatar
GET	/api/gamification/badges	Catalogue de badges
POST	/api/gamification/teams/{groupId}/points	Ajouter des points
GET	/api/gamification/leaderboard	Classement

Interactions (autres services)

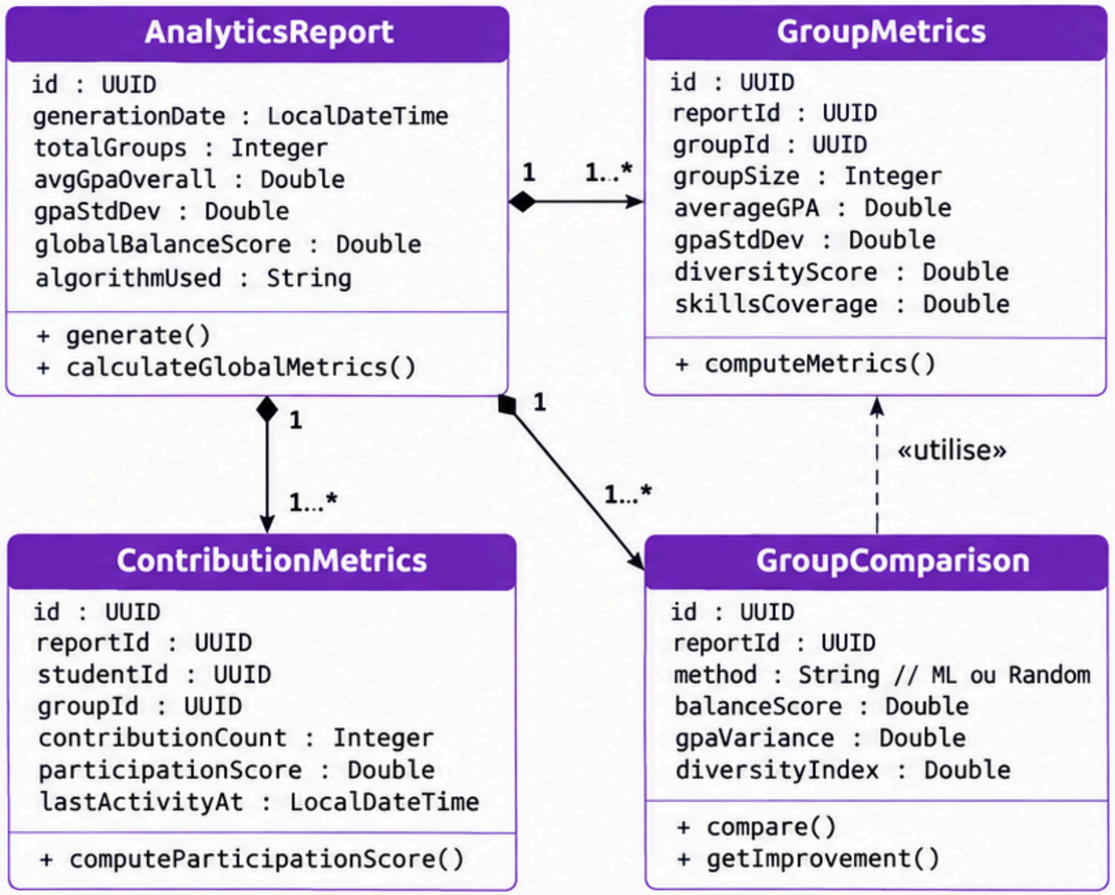
Consomme	Consomme GroupValidated (RabbitMQ) → crée TeamIdentity + avatar
Publie	Événement BadgeAwarded → RabbitMQ (notifications)
REST / Feign	Route Gateway : /api/gamification/** → :8085
Infra	Eureka :8761 + Config Server :8888

MS6 · Analytics Service

MS6

Responsable : Amen Bannour Spring Boot · port 8095 · H2 (analytics_db)

Diagramme de classes



Endpoints REST

POST	/api/analytics/reports	Générer un rapport d'analyse
GET	/api/analytics/reports/{id}	Consulter un rapport
GET	/api/analytics/reports/{id}/groups	Métriques des groupes
GET	/api/analytics/reports/{id}/contributions	Participation des étudiants
GET	/api/analytics/indicators	Indicateurs globaux (écart, diversité, …)
GET	/api/analytics/compare?reportId={id}	Comparer avec une répartition aléatoire
POST	/api/analytics/compute	Recalculer les indicateurs d'un groupe

Interactions (autres services)

REST / Feign	Récupère les groupes et leurs membres depuis le Course/Group Service (MS3)
REST / Feign	Récupère les profils étudiants (GPA, compétences, styles, intérêts) depuis le Profiling Service (MS2)
Consume	Événement *GroupsCreated* → déclenche le calcul des indicateurs
Publish	Événement *AnalyticsComputed* → met à jour les tableaux de bord (front-end)
Infra	Enregistré dans Eureka :8761, configuration via Config Server :8888, accès via API Gateway :8080

Conclusion

Merci pour votre attention